TECHNICAL DESIGN TASK · SEPTEMBER 2026

Helix Platform *Architecture*

A multi-tenant platform built to host 100+ integrated products — search, meetings, calendar, storage, cloud services — for global users. Modular monolith now; services when a measured trigger says so, not before.

STACK

TypeScript · NestJS · Next.js · PostgreSQL · Redis

DEPLOYABLES

API · worker · media plane

SAMPLE PROJECT

4 products, running & seeded

EVIDENCE

38 tests + 16 live checks, transcripts included

> A platform hosting 100+ products does not need 100 microservices. It needs boundaries a product *cannot* violate, and a kernel a product *cannot* bypass.

This design takes the boundaries on day one — one Postgres schema per product, events as the only cross-product dependency, one declarative manifest per product, a URL prefix per product — and defers the *distribution* until a written, measurable trigger justifies paying for it.

Two consequences follow, and everything below is in service of them: adding product #101 is a single command, and extracting any product into its own service is a deployment change rather than a rewrite.

01 Which architecture should be used?

The boundaries a microservice architecture gives you are valuable. The distribution is what costs you. So: take the boundaries now, buy the distribution later, per product, on evidence.

PART	NOW (START)	FUTURE (SCALE)	NOTES / REASON
Backend	**Modular monolith** — one deployable, N product modules behind a manifest contract. Worker and realtime media are already separate.	**Service-based → microservices**, extracted per product as triggers fire. Then **cell-based**: each region runs a full stack for a slice of tenants.	One deployable means one transaction, one deploy, one stack trace. The module contract makes extraction a move, not a rewrite. Worker and media are split on day one because their scaling profile genuinely differs.
Admin	**Separate Next.js app**, same API, elevated permission set.	Same app hardened — private network, SSO, mandatory MFA; splits into ops and support consoles when the audiences diverge.	Blast radius. An XSS or logic bug in an internal tool must not be reachable from a customer session. The split costs almost nothing because it shares the design system and SDK.
Frontend	**One Next.js app** — platform shell (nav, auth, launcher, global search) plus feature-based routes per product.	**Micro-frontends** via Module Federation: the shell stays, each product UI deploys independently.	With 3–20 products one app is faster. The trigger is not product count — it is the shared deploy pipeline becoming the bottleneck at roughly ten frontend teams.
API layer	**One versioned REST API** `/v1/{product}` with OpenAPI. Gateway concerns are global guards.	**Dedicated gateway** (Envoy/Kong), **BFF per client**, **GraphQL federation** for aggregation, **gRPC** between services.	Clients are coded against `/v1/calendar/…` from day one. When Calendar moves to its own service the gateway re-points that prefix and no client changes. The URL shape is what buys the migration.
Database	**One PostgreSQL cluster, one schema per product** + a `platform` schema. Row-Level Security on every tenant-owned table.	**Database per service**, tenant **sharding** (Citus/Vitess) for the largest products, **CQRS** read models, per-region clusters.	Schema-per-product means a product's tables are already disjoint. Extraction is a `pg_dump --schema=` and a new connection string — not an untangling of foreign keys.

When a product leaves the monolith

A product is extracted when at least one of these is *measurably* true. Nothing else counts — and specifically, “it feels big” and “microservices are modern” do not.

TRIGGER	MEASURABLE THRESHOLD
Divergent scaling	Resource-per-request differs from the platform median by >5×, or it needs hardware the monolith has not (GPU, high bandwidth, stateful media).
Fault isolation	Its failure modes threaten the platform SLO — it can exhaust connections or memory shared with 99 others.
Team contention	More than two teams on one pipeline, or median PR-to-production >30 min because of unrelated tests.
Cadence conflict	It must ship hourly while the platform ships daily, or needs a maintenance window the platform cannot take.
Compliance boundary	Its data must sit in a separate jurisdiction or certification scope that would otherwise widen the audit boundary to everything.
Runtime mismatch	It needs a runtime the monolith cannot host — ML inference, video transcoding, a native SDK.

Two components are *born extracted* because they meet a trigger on day one: the **worker** (long jobs starve short requests) and the **media plane** (stateful, bandwidth-bound, different runtime). That is the trigger list applied honestly, not an exception to it.

Handling 100+ products

One declarative manifest

Each product declares key, DB schema, API prefix, permissions, published and subscribed events, search documents, quotas and plan availability. Routing, RBAC, indexing, billing and the app launcher are all derived from it. There is no second place to register a product.

Boot-time graph validation VERIFIED

Before a request is served, the registry checks every manifest together: duplicate keys, schemas or prefixes, permission collisions, permissions outside a product's namespace, events subscribed to that nobody publishes. Any error refuses the boot.

A generator VERIFIED

`pnpm gen:product notes` scaffolds the manifest, module, controller, service and docs, and wires the composition root. In this repo it produced a fourth product that booted, routed and synced permissions with zero manual edits.

A kernel that keeps products small

Auth, tenancy, RBAC, audit, rate limiting, search, notifications, storage and quotas live in the kernel, so a product is only its domain logic. A cross-cutting fix ships once for 100 products instead of 100 times.

02 What will the folder structure be like?

PART	NOW (START)	FUTURE (SCALE)	NOTES / REASON
Backend	One app: `src/platform/` (kernel), `src/products/{key}/` (one folder per product), `src/shared/` (infrastructure).	Each extracted product becomes `services/{key}/` with the same internal layout; the kernel becomes `packages/platform-kernel/`.	A product folder is already a complete, self-contained unit. Extraction is `git mv`, not reorganisation.
Admin	`apps/admin/` — separate Next.js app sharing `@helix/ui` and `@helix/sdk`.	Same shape; splits into `admin-ops/` and `admin-support/` when the audiences diverge.	Separate deployable from day one for blast radius. Never a separate repo — sharing the design system matters more.
Frontend	`apps/web/` — shell in `src/app/`, product UIs in `src/app/apps/{key}/`, cross-cutting logic in `src/features/`.	Each product UI becomes `apps/web-{key}/` exposing a federated remote; `apps/web-shell/` hosts them.	Product UIs are already isolated by route folder, so the micro-frontend split is a move plus a federation config.

THE TREE AS IT STANDS

```
apps/
  api/
    prisma/          schema.prisma · migrations · sql/ (RLS + isolation proof) · seed.ts
    src/
      app.module.ts  composition root — the security model, readable top to bottom
      platform/      THE KERNEL — true for all 100+ products
        auth/ rbac/ tenancy/ rate-limit/ audit/ events/
        search/ storage/ notifications/ feature-flags/ registry/ health/
      products/      ONE FOLDER PER PRODUCT — 100+ of these
        calendar/    manifest · module · controller · service · dto/ · README
        meet/  drive/  notes/            notes/ was generated
      shared/        config · prisma · redis · http (guards, filters, decorators)
  web/               platform shell + one route folder per product UI
  admin/             internal console — separate deployable
  worker/            outbox relay · queue consumers · scheduled maintenance
packages/
  core/              THE CONTRACT — manifest, EventBus, tenant context, registry
  sdk/ ui/ config/   typed client · design system · the cross-product import ban
infra/               docker-compose · kubernetes · terraform
tools/generators/    pnpm gen:product — product #101 in one command
docs/                the six answers · ADRs · verification transcript
```

The future tree is *the same tree*: folders move up a level and gain a `package.json`. Nothing is renamed or re-layered. That is only true because the boundaries were real from the start.

Separation of concerns

Three directories, three reasons to change: kernel, domain, infrastructure. A product folder contains no infrastructure and no security code.

Feature-based, never layer-based

A tree of `controllers/ services/ models/` makes every product change touch every directory — and makes extraction impossible.

Machine-enforced boundaries

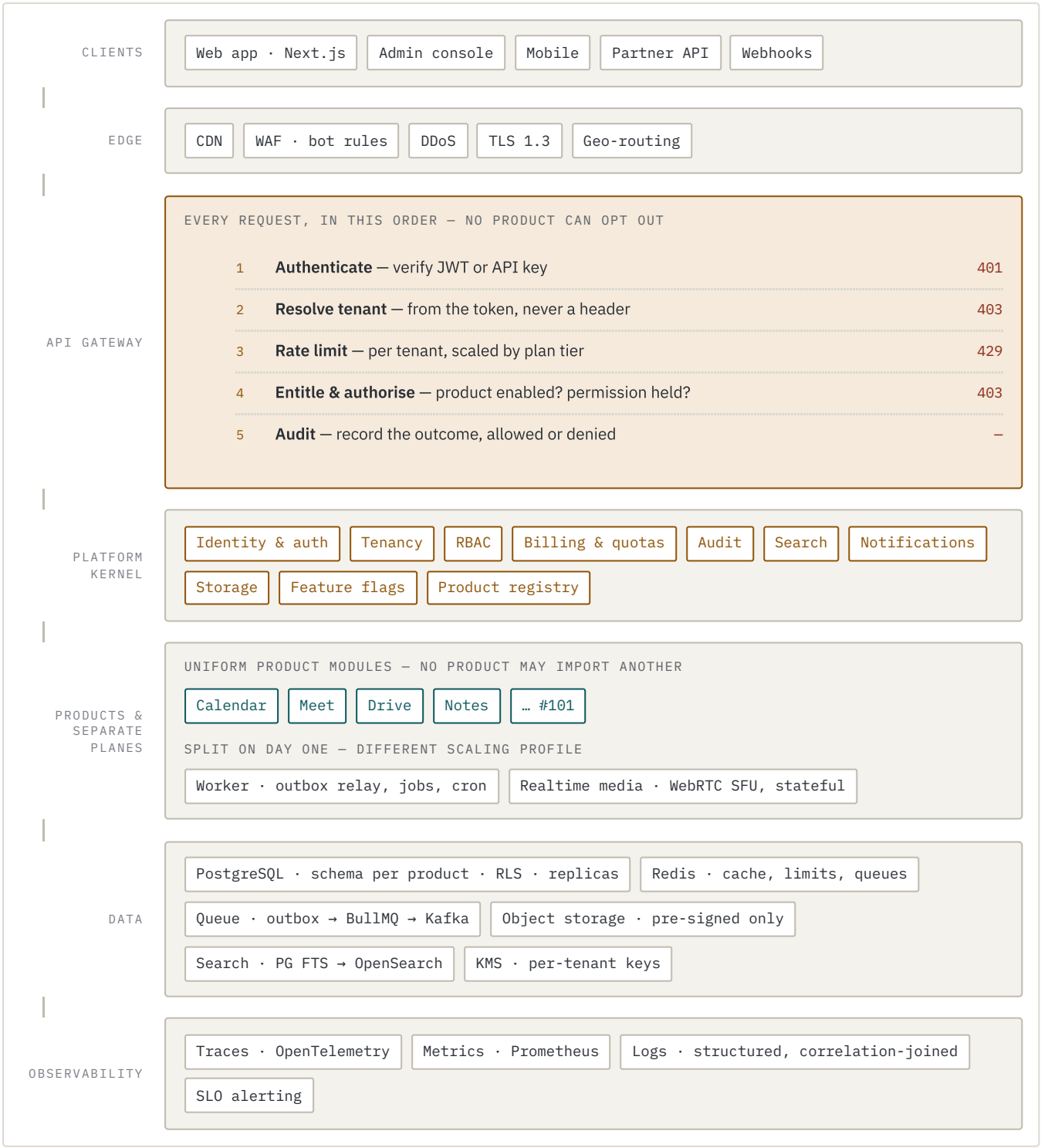
An ESLint rule forbids cross-product imports; per-schema grants forbid cross-product reads; the registry forbids namespace collisions. Boundaries kept by good intentions do not survive 100 products.

Docs that travel with the code

Every product ships a README with its boundaries and its extraction checklist, so the knowledge is not in one architect's head.

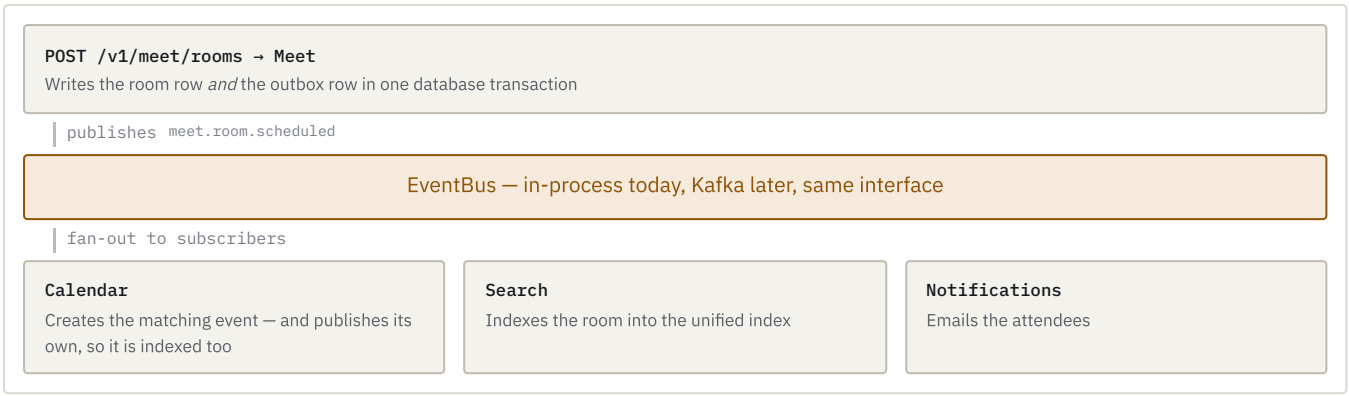
03 System design

Users → frontend → API layer → backend services → database, cache, queue and storage — with every request passing the same five gates before any product code runs.



How products cooperate without coupling

This is the mechanism that makes 100 products behave like one product — and the reason any of them can be extracted later without the others changing.



Meet does not import Calendar. Calendar does not import Meet. Neither knows Search or Notifications exist. The **transactional outbox** makes it safe: the domain row and the event row commit together, so no event exists for a change that rolled back, and no committed change fails to publish.

What each major component is for

API gateway

The single front door. Putting authn, tenancy, rate limiting, entitlement and audit here means a product *physically cannot ship without them* — the decisive property when many teams contribute.

Authentication

Short-lived stateless access JWT keeps the hot path free of session lookups; server-side rotating refresh sessions keep revocation instant. Reuse of a revoked token revokes every session for that user.

Authorization / RBAC

Permissions are resolved at token-mint time and cached, so authorising a request is a string comparison, not a three-table join. At 50k req/s that is the whole difference.

Product registry

Loads and validates the whole manifest graph at boot. The guardrail that keeps 100 products coherent without central review of every change.

PostgreSQL

System of record. Schema per product for isolation; RLS for a second, independent enforcement of tenant boundaries; replicas for read-heavy products and reporting.

Redis

Cache, rate-limit counters and queues, all tenant-prefixed. Caching tenant context and permissions removes the two queries that would otherwise sit on 100% of requests.

Object storage

Bytes never pass through the API — clients get a pre-signed URL. A 5 GB upload consumes no application capacity. Keys are prefixed per tenant, so deletion, encryption and lifecycle are prefix operations.

Unified search

Users search for a thing, not for the product that owns it. Products publish events and never talk to a search engine, so Postgres FTS → OpenSearch is a one-file change.

Notifications

One pipeline for in-app, email, push and webhook. Preferences, quiet hours, digests and unsubscribes are built once — and a badly-behaved product cannot spam a customer.

Worker

Separate from day one: long jobs and short requests cannot share a process without the slow work eventually starving the fast work.

Audit log

Append-only; `UPDATE` and `DELETE` are revoked from the app role at the database. Written by kernel interceptor and exception filter, so coverage cannot regress when a team adds an endpoint.

Feature flags

Percentage rollout, per-tenant enablement, and kill switches — the fastest lever in an incident, because it disables an expensive path without a deploy.

04 What technology will be used?

PART	NOW (START)	FUTURE (SCALE)	NOTES / REASON
Backend	**TypeScript + NestJS** (modular monolith), Prisma, Node 22	Same for most products. **Go** for extracted hot paths, **Rust/C++** for the media SFU, **Python** for ML products	NestJS's module, DI and global-guard model *is* the product-module contract — the kernel is expressible in the framework rather than fought against it. Polyglot only where a trigger justifies it: the event contract is JSON, so language becomes a per-service decision.
Admin	**Next.js + React 19**, shared design system	Same, plus SSO/OIDC, mandatory MFA, IP allow-list; ops and support consoles split	Reusing the customer stack means one hiring profile and one design system. It stays a separate *deployable*, not a separate stack.
Frontend	**Next.js 15 App Router**, React 19, server components for data-heavy pages	**Module Federation** micro-frontends; edge rendering per region	Server components keep tokens server-side and cut client JS — it matters when a launcher may list 100 products. The route-folder model is already the micro-frontend seam.
API layer	REST + **OpenAPI 3.1**, URI versioning per product, generated typed SDK	**Envoy/Kong**, **gRPC** internally, **GraphQL federation** for aggregation, WebSocket/SSE	REST first because it is CDN-cacheable, debuggable with curl and consumable by partners. gRPC and GraphQL arrive where they earn their cost.
Database	**PostgreSQL 17** — schema per product, RLS, PgBouncer, one read replica	**Citus/Vitess** sharding, DB per service, **ClickHouse** for analytics, per-region clusters	Postgres covers relational integrity, JSONB, full-text search, partitioning and `SKIP LOCKED` queues well enough that one engine covers years one to three.

What is deliberately *not* adopted yet

Every row is a cost avoided today with a written condition for paying it later. This table is as much the architecture as the components that were chosen.

NOT ADOPTED	WHY NOT YET	TRIGGER TO ADOPT
Microservices	Highest price for the decision the team is least qualified to make: where the boundaries are.	Any product meets an extraction trigger.
Kafka	BullMQ handles job processing; Kafka's value is replay and multiple consumer groups.	First service extraction, or a consumer needing replay.
Separate search cluster	Postgres FTS is genuinely good to a few million documents per tenant.	>5M docs per tenant, or relevance Postgres cannot express.
Service mesh	A control plane, sidecar latency and a debugging surface — for three deployables.	>10 services needing mTLS and traffic policy.
GraphQL	REST + OpenAPI already generates a typed SDK and caches at the CDN.	A client aggregating across ≥3 products per round trip.
Multi-region active-active	Doubles operational complexity for a single-region customer base.	A residency clause, a latency SLO, or a costed outage.

05 Security & scalability

Items marked VERIFIED were executed against the running system; the transcripts are below.

SECURITY

Authentication

scrypt with per-password salt. 15-minute access JWT for a stateless hot path; opaque 30-day refresh token, hashed at rest, rotated on every use. Replay of a revoked token revokes every session for that user. Login does comparable work whether or not the account exists.

Authorization / RBAC VERIFIED

user → membership → roles → permissions, keyed `product.resource.action` and declared by manifests. Routes are protected by default; opting out needs `@Public()`. A `member` was refused `calendar.event.delete`; the owner succeeded.

Multi-tenancy VERIFIED

Three layers: tenant from the token never a header; `tenantId` on every query; Postgres RLS as the backstop. A forgotten filter returns *zero rows*, not another tenant's data.

API security

TLS 1.3, Helmet headers, CORS allow-list, whitelisting validation so mass-assignment is impossible, parameterised queries throughout, one RFC 9457 error shape, per-product versioning.

Quotas VERIFIED

Per-plan limits are declared in each product's manifest and enforced by the kernel. The product names a metric; the kernel resolves the tier, finds the limit and refuses with `402`. Counters roll back on refusal, so two concurrent requests at the limit cannot both pass.

Rate limiting VERIFIED

Per-tenant sliding window in Redis, one atomic round trip, scaled by plan tier. Per tenant rather than per IP, because tenants share NATs and the resource protected is capacity per paying customer.

Session handling VERIFIED

Web tokens live in httpOnly cookies set by a server action, never in `localStorage` — an XSS in any one of 100 product UIs must not be able to read the token and impersonate the user across every other product.

Data encryption

TLS in transit, volume encryption at rest, envelope encryption with per-tenant KMS keys — so destroying one key crypto-shreds one customer and makes a deletion request provable.

Secure file storage

Pre-signed URLs only, MIME allow-list, size caps, async malware scanning that gates download, private buckets, versioning and object lock.

Audit logs VERIFIED

Append-only, written by the kernel, covering *denials as well as successes* — a trail that records only what succeeded is blind to an attacker probing for permissions they lack.

Backup & recovery

Snapshots plus continuous WAL archiving for PITR to any second in 35 days. RPO ≤ 5 min, RTO ≤ 1 h. Restores tested on a schedule — an untested backup is a hypothesis.

SCALABILITY

Database scaling

Cheapest first: indexes leading with `tenantId` and cursor pagination, pooling, read replicas, partitioning, vertical scale, then schema→database split, then tenant sharding, then CQRS read models.

Caching

CDN, HTTP `ETag`, application cache for tenant context and permissions, per-product query cache. Correctness comes from event-driven invalidation, not from short TTLs alone.

Load balancing

Geo-DNS to the nearest healthy region; L7 least-outstanding-requests across zones; removal driven by `/readyz` so a replica with a sick dependency drains instead of erroring.

High availability

≥6 replicas, zone spread, PDB at 75%, `maxUnavailable: 0`, synchronous DB standby. Products declare soft dependencies and must degrade gracefully — scheduling a Meet room still succeeds if Calendar is down.

Disaster recovery

Tiered objectives (auth ≤15 min, standard products ≤1 h, analytics ≤24 h), regions as Terraform modules, rehearsed runbooks, cell-scoped blast radius, kill switches.

Async everywhere slow

Anything over ~200 ms is a queue job, and uploads bypass the API entirely. The request path stays fast because nothing slow is allowed on it.

06 Future architecture

The stages are triggered by measurements, not by calendar time or product count. A platform can sit at stage one with forty products and be entirely correct.

01 Modular monolith

`~10 products · <10k tenants · 1 region`

One API, one worker, one media plane, one Postgres, one Redis, Postgres full-text search. Every boundary that later extraction depends on already exists.

02 Distributed services

`Trigger — the first product meets an extraction threshold`

Search, notifications and the busiest product move out first. Kafka replaces BullMQ as the backbone; the `EventBus` interface, and therefore every product, is unchanged. The gateway re-points prefixes and clients notice nothing. The other ~25 products stay put.

03 Selective microservices

`Trigger — team topology, more than load`

~15 services, each owned end-to-end. Database per service; cross-service reads become read models built from the event stream. Schema registry, sagas, micro-frontends, service mesh. This is where the architecture gets *harder* — worth it only because 60 products and 15 teams on one deployable is worse. The long tail stays in the monolith permanently.

04 Global infrastructure

`Trigger — a residency clause, a latency SLO, or a costed outage`

Cell-based topology: a cell is a complete stack serving a slice of tenants, so a failure is scoped to a cell and a bad deploy rolls out cell by cell. Tenant data stays in its home region; identity metadata replicates everywhere.

05 Platform as a product

`Trigger — external developers, or a marketplace`

The product manifest — the internal contract from day one — becomes the public extension contract. Third-party products declare permissions, events, quotas and UI entry points exactly as internal ones do. This is the design's long-term payoff.

Why the evolution is credible

Four decisions made at stage one, none requiring anyone to predict the future:

DECISION MADE NOW	WHAT IT BUYS LATER
Products own a schema, never read another's	Extracting a service's data is a schema dump, not a data migration
Cross-product traffic only via a named-event interface	Swapping in-process for Kafka is one class; product code untouched
Clients address products by URL prefix	The gateway re-points a prefix; no client ever changes
A kernel products cannot bypass	A security fix ships once for 100 products, before and after the split

None of these is expensive now. All are close to impossible to retrofit at stage three. **That asymmetry — cheap now, impossible later — is the only sound reason to do architectural work ahead of need**, and it is the criterion every decision here was measured against.

– Verification

A design document that implies more than it demonstrates is worse than one that admits its edges. These ran against the live system.

PASSED TENANT ISOLATION IS ENFORCED BY THE DATABASE, NOT JUST BY CODE

```
-- session is tenant A; the query has NO tenant filter at all
SELECT title FROM calendar.events;
  SECRET-A                          (1 row — tenant B's row is invisible)

-- explicitly asking for another tenant's rows
SELECT count(*) FROM calendar.events WHERE "tenantId" = '<tenant-A>';
  leaked_rows: 0

-- writing a row for another tenant
  NOTICE: PASS: cross-tenant insert blocked by RLS policy

-- and with no tenant context set at all
  rows_visible_without_tenant: 0   ← zero rows, never "all rows"
```

PASSED CROSS-PRODUCT BEHAVIOUR WITH ZERO COUPLING

```
POST /v1/meet/rooms  {"title":"Quarterly platform review"}
  Meet created room ayu-jusp-unu

GET /v1/calendar/events
  Quarterly platform review        <- created by the Meet event

GET /v1/platform/search?q=platform+review
  [calendar] calendar.event   Quarterly platform review
  [meet    ] meet.room        Quarterly platform review
```

PASSED RBAC, ENTITLEMENT AND THE AUDIT TRAIL

```
member  DELETE /v1/calendar/events/{id}
  403 forbidden — Missing permission: calendar.event.delete
owner   DELETE /v1/calendar/events/{id}
  {"deleted": true}

-- with the Meet product disabled for this workspace
POST /v1/meet/rooms
  403 product not enabled — The "meet" product is not enabled for this workspace

-- the kernel recorded all of it, denials included
 product  | action                          | outcome | reason
----------+---------------------------------+---------+--------------------------------
 meet     | POST /v1/meet/rooms             | denied  | The "meet" product is not enabled
 calendar | DELETE /v1/calendar/events/:id  | allowed |
 calendar | DELETE /v1/calendar/events/{id} | denied  | Missing permission: calendar.eve…
 meet     | POST /v1/meet/rooms             | allowed |
```

PASSED PRODUCT #101 IS A COMMAND — AND A BAD MANIFEST CANNOT START THE PLATFORM

```
$ pnpm gen:product notes --name "Helix Notes" --category productivity
  Created product "notes" — manifest, module, controller, service, README
  Registered in app.module.ts and product-registry.service.ts

-- next boot, with zero manual edits:
  [RouterExplorer] Mapped {/v1/notes/items, GET} route
  [RouterExplorer] Mapped {/v1/notes/items, POST} route
  [registry] Registered 4 products, 15 permissions

-- then: make "notes" declare a permission belonging to "calendar"
  ERROR [registry] [notes] Permission "calendar.event.read" must be namespaced under "notes."
  ERROR [registry] [notes] Permission "calendar.event.read" already declared by "calendar"
  Error: Product registry validation failed with 2 error(s). Refusing to start.
```

PASSED QUOTAS ENFORCED FROM THE LIMITS A PRODUCT DECLARES

```
business plan allows 100,000 calendar events/month (from calendar.manifest.ts)

POST /v1/calendar/events   # event #100,000
  created: At the limit

POST /v1/calendar/events   # event #100,001
  402 quota exceeded — Monthly eventsPerMonth limit reached for calendar on the business plan

  counter after refusal: 100000   (rolled back, no drift)
```

WHAT IS NOT VERIFIED

Load and failover

No load test, chaos test or failover drill was run. The scaling figures are design targets, not measurements.

Kubernetes, Terraform, multi-region

Manifests and the Terraform cell module are written and reviewed, but never applied to a cluster or a cloud account. Kafka, OpenSearch, sharding and cells are the future column throughout — nothing here runs them.

The media plane

Designed and justified, with its data model in place, but not implemented.

No end-to-end test suite

38 automated tests cover the registry rules, RBAC wildcard matching, password hashing and the product-manifest contracts. Behaviour that only exists once the system is running is covered by `verify.sh` transcripts, not by tests that would fail CI.

Remaining hardening

Most product reads rely on the application-level tenant filter with RLS as the backstop; connecting the API as the restricted `helix_app` role everywhere is the outstanding step. The worker's quota-flush job is written but not exercised here.

RUN IT

`pnpm infra:up && pnpm db:migrate && pnpm db:seed`, then `./scripts/verify.sh` to reproduce every transcript above.

READ IT

Six numbered answers in `docs/`, five decision records in `docs/adr/`, and the full transcript in `docs/VERIFICATION.md`.

THE SINGLE IDEA

Boundaries on day one; distribution on evidence.
Everything else follows from that.